

Rogue AI Detection: Build Plan for the MSP Pitch

Prepared: Sep 30, 2026 for Darryl. **Target:** working lab demo in front of MSPs in 4 to 6 weeks. **Builds on:** AI Threat Detection for MSPs: Architecture and MVP Plan.

1. What you are building, in one paragraph

A sensor you place in a client network that answers three questions an MSP's client will actually ask: **(1)** are staff pasting company data into AI tools, **(2)** is someone attacking us with AI-speed automation, and **(3)** are any autonomous agents running on our systems that we did not approve. Deterministic rules and simple baselines do the detecting. An AI triage agent reads redacted case summaries and explains them in plain English. It is read-only and never blocks anything in the MVP.

The pitch is **AI visibility and risk detection**, not "we catch all AI." Every claim below is paired with what the product cannot see, because an overclaim that fails in front of an MSP costs more than a smaller claim that holds.

2. The three goals as detection tracks

Goal	What the rules look for	Data needed	Can't see	Demo proof
1. Data pasted into AI tools	Visits to a maintained AI-domain list; paste-sized and file-sized POSTs to those domains; bursts of uploads; OAuth grants to AI apps	Proxy/DNS/firewall logs; M365 or Google audit logs	Personal devices, home networks, message content (size and direction only)	jsmith uploads 480 KB to DeepSeek and sends 7 paste-sized posts to ChatGPT; carol only browses and is not flagged as an upload
2. AI-speed attack	Median gap between logins under 1s; low variance in timing; password spray across many accounts; wide-and-fast scans; fail-then-success	Auth logs, firewall or NetFlow	Proof the attacker is an AI. We report automation-like behavior with a confidence level	External IP sprays 30 accounts at machine speed, scans the LAN, then logs in as a service account: critical case
3. Unapproved agents	Ollama, LM Studio, llama.cpp, MCP servers, agent frameworks, coding agents; model files (.gguf, .safetensors); listening ports; sustained GPU; server calls to AI APIs	Endpoint inventory (agent or scheduled script); egress logs	Unmanaged or off-network machines; agents inside an approved tool	DEV-LAPTOP-07 runs Ollama plus an MCP server with a 4.9 GB model; SRV-APP-02 calls an AI API 45 times

Working code for all three already exists in the repo scaffold that comes with this plan: 5 automated tests pass, and the demo run flags exactly the injected scenarios while leaving normal users alone. That gives you a real, repeatable demo on day one rather than slides.

3. Six-week roadmap

Week	Focus	Exit criteria
1	Get the scaffold running locally and into GitHub. Stand up a lab: two Windows VMs plus one Linux VM as the sensor. Install Zeek and Wazuh on the sensor.	Repo pushed; demo runs; Wazuh sees both VMs; Zeek produces DNS and conn logs from the VM traffic

Week	Focus	Exit criteria
2	Goal 1 on real traffic: loader that turns Zeek/proxy logs into the detector's columns. Generate real ChatGPT/Claude/DeepSeek browsing and paste tests from a VM. Tune thresholds.	Goal 1 fires on your own live traffic, with the false positives you saw written down
3	Goal 3 on real endpoints: run collect_inventory.ps1 (or the Wazuh agent) on the VMs; install Ollama and a small MCP server as the 'rogue' scenario. Approved-software list per host.	Ollama and MCP server detected within one collection cycle; an approved one is suppressed
4	Goal 2 on real logs: Windows failed-logon events via Wazuh; run a controlled spray and scan from another VM using a tool you own (for example Hydra or Nmap) against your lab only.	Spray and scan flagged with a confidence level; normal logins not flagged
5	Triage agent and console: turn on cloud triage with redaction, audit log, and per-client off switch. Build a one-page case view and PDF findings report. Measure API cost per case.	Report reads well to a non-technical client; cost per case is a measured number
6	Pitch readiness: 10-minute demo script, AI Exposure Assessment one-pager, coverage-limits slide, pricing hypothesis, list of 10 target MSPs.	Two dry runs done; three MSP conversations booked

Weeks 1 to 4 are mostly evenings-and-weekends work if you build solo. If you decide to add a contract developer, the cleanest handoff is the console UI in week 5; keep detection logic yourself because it is your differentiator.

4. Lab architecture for the demo

One Linux sensor VM with a virtual span port sees traffic from two Windows client VMs and an 'attacker' VM. Zeek writes DNS, HTTP and connection logs. Wazuh manager collects Windows events and endpoint inventory. A small Python loader normalizes both into the CSV columns the detectors read. The detectors produce findings, the triage layer groups them into cases and redacts usernames, hostnames and IPs, and the console shows cases per goal. Everything except the optional cloud call runs locally.

Layer	Use	Build or reuse
Network visibility	Zeek and Suricata on a span port or tap	Reuse
Endpoint and logs	Wazuh manager and agent	Reuse
Goal detectors and baselines	Python, already scaffolded	Build (your differentiator)
Triage agent	Claude API, read-only, audited, redacted	Build
MSP console and PDF reports	FastAPI plus a simple front end, or a C# service if you prefer	Build
Packaging	Docker Compose, later an OVA	Build

Confirm the Wazuh, Zeek and Suricata licenses fit how you plan to resell before committing.

5. Recommendations for AI intrusion detection

Detect behavior, not identity. Report machine-speed timing, uniform sessions and breadth of probing as automation-like activity with a confidence level. Never write 'an AI attacked you'.

Treat the AI domain list as a product. Version it, publish changes to every client at once, and let each client mark tools as sanctioned. New AI tools appear weekly; this file is where you stay ahead of larger vendors.

Add an approved-agent registry per client. Goal 3 is only meaningful against a list of what is allowed. The registry (host, tool, owner, reason) doubles as an onboarding step that sells itself to the client's IT lead.

Treat log text as hostile. Usernames, file names and user agents can carry prompt-injection text. The triage prompt already tells the model to treat case content as data, and the agent has no write tools.

Keep humans in the loop. No auto-blocking in the MVP. Output goes to a ticket or dashboard the MSP's technician owns.

Guard the sensor itself. It has read access to a client network. Use per-client credentials, least privilege, signed updates, and a short written security whitepaper before your first pilot.

Later, not now: a browser extension inventory, DLP-style content inspection, auto-remediation, and integration with ConnectWise, N-able or Datto ticketing. Add these only when a pilot asks.

6. The MSP pitch

Demo script (about 10 minutes)

Min	Show	Message
0-2	The three client questions, then the coverage-limits slide	We promise visibility we can defend, not total coverage
2-4	Goal 1 case: jsmith and DeepSeek	Plain-English finding, size-based evidence, recommended action
4-6	Goal 3 case: Ollama and MCP server on a dev laptop	Approved list vs. found list; the client's IT lead has never seen this
6-8	Goal 2 case: spray, scan, service-account login	Automation-like, confidence high; what to do in the next hour
8-10	Cost and control: cloud switch, redaction, audit log	Works on rules alone; nothing raw leaves the client network

Lead offer: the AI Exposure Assessment. Two weeks of sensor time, then a report of which AI tools staff use, what upload activity looks like, and which agents or local models exist. It is low risk for the client, easy for the MSP to sell, and converts to a recurring service. Your pre-sales background fits: run each pilot like a POC with a discovery call, a written success list and a findings readout.

Design partners. Start with two or three friendly MSPs on a free 60-day pilot in exchange for feedback and a reference. Sources: your Ivanti and ITSM network, Bay Area MSP peer groups, and ConnectWise, N-able and Datto communities. **Pricing is a hypothesis:** check what comparable MSP security add-ons charge, and price from your measured cost per client (hosting, LLM usage, support time) after the pilot.

7. Decisions and risks to settle early

Item	Why it matters	Action
Employee monitoring law	Monitoring staff traffic and endpoints is regulated; California has notice requirements	Written monitoring policy and employee notice template for clients; lawyer reviews contract and data-processing terms
Liability and insurance	A missed detection can become a blame dispute	E&O; and cyber insurance; liability limits in the contract with a lawyer
False positives	Alert fatigue kills MSP adoption faster than misses	Track precision from the first lab run; tune before any pilot
Data handling	Logs contain sensitive data; MSPs will ask about SOC 2	Redact before cloud calls, document storage, publish a whitepaper; certify later
Competition	Larger vendors are adding AI-usage visibility	Stay narrow, fast and MSP-friendly; integrate with RMM and ticketing
Attack testing	Lab attacks must stay in your lab	Only run spray and scan tools against systems you own

8. What to do next

1) Run **deploy_local.ps1** to copy everything into your project folder. 2) Run **push_to_github.ps1** to publish it to your repo. 3) Run the demo locally (README quick start). 4) Start week 1 of the roadmap: build the lab VMs. Send me the first real Zeek or Wazuh log samples and I will write the loaders that feed the detectors.